



ASSUMPTION UNIVERSITY

Vincent Mary School of Science and Technology

CSX4104 / ITX4104 Software Testing

Term Project

By

Lut Lat Aung - 6511163

Lin Myat Phyo - 6530201

Thet Myat Noe Thwin - 6530213

Khaing Thin Zar Sein - 6530381

Khin Myo Aye - 6540211

Table of Contents

Introduction.....	2
System Overview.....	2
System Scope and Requirements.....	2
System Scope.....	2
System Requirements Specifications.....	2
Functional Specifications with Pseudo-Code Mapping.....	4
Loan Estimation Function.....	4
Description.....	4
Input and Validation.....	4
Process.....	5
Output.....	6
Interest Calculation Function.....	6
Description.....	6
Input and Validation.....	8
Process.....	9
Output.....	10
Control Flow Graphs and Data Flow Graphs for Loan Estimation.....	10
Control Flow Graphs and Data Flow Graphs for Interest Calculation Functions.....	12
Input domains.....	16
Software Testing Approach.....	18
Test Cases and Scenarios.....	18
Loan Estimation Function Test Cases.....	18
Interest Calculation Function Test Cases.....	19
Test Results and Analysis.....	22
User Interface Testing.....	24
Conclusion.....	29

Introduction

This report presents the software testing conducted on a small-scale offline system designed for a gold pawn shop business. The system aims to replace traditional handwritten pawn slips and manual financial calculations with a software-based solution to enhance efficiency, accuracy, customer satisfaction, and security. The report focuses on testing two critical functions of the system: **Loan Estimation** and **Interest Calculation**.

System Overview

The pawn shop system consists of three main subsystems:

1. **Pawning Process**: Facilitates the pawning of jewelry in exchange for loans.
2. **Redeeming Process**: Allows customers to reclaim their pawned items by repaying the loan amount plus any accrued interest.
3. **Generating Reports**: Provides daily and monthly reports to offer insights into the shop's financial performance.

System Scope and Requirements

System Scope

The primary focus or the system scope of this project is on the **Loan Estimation** function from the pawning process and **Interest Calculation** function from the redeeming process. These functions are critical for calculating the loan amount the user can offer based on the weight and purity of the pawned items and computing the total interest owed upon item redemption, respectively. As for other functions outside of our system scope like pawn slip registering or generating reports, we've just done some mock-up user interfaces.

System Requirements Specifications

Here, shop owners are considered as users

Loan Estimation - Users can obtain an estimation of the loan amount to offer by entering the weight of the jewelry in grams and selecting its purity.

R_1.1 : When the user puts in a negative number or zero as gold weight, it shows an error.

R_1.2 : When the user puts in a positive valid gold weight, it correctly retrieves the gold price of selected purity and calculates the estimated loan amount to be offered.

Interest Calculation - Users can type in the slip ID to calculate the total interest the customer will have to pay. The system will automatically determine the applicable interest rate based on predefined loan amount ranges, eliminating the need for users to input the rate manually. The interest calculation will consider the period from the day the item was pawned to the current date as loan period. 30 days is considered to be one month. The system will enforce the following rules for interest calculations.

R_2.1 : When the user puts in the pawn slip ID, it checks whether the ID exists in the database (in our case, mock-up Json file)

R_2.2 : If the pawn slip ID exists, it checks if it is already paid or not. If it is not paid yet, it checks other conditions and calculates the interest.

R_2.3 : If a pawn slip is marked paid, it display a message saying “already paid”

R_2.4 : If also checks if the loan amount is valid.

R_2.5 : If loan amount is valid and less than 5000, it automatically sets the interest rate to 4%

R_2.6 : If loan amount is less than 10000, it sets the interest rate to 3.5%

R_2.7 : If loan amount is more than or equal to 10000, it sets the interest rate to 3%

R_2.8 : If the calculated loan period is less than or equal to one month (30 days), the loan period will be considered as one month.

R_2.9 : After the first month, if the loan period is within the first 7 days of each month, it will be considered valid for half month interest. For example; on the day of redemption, if a pawn slip has a loan period between 30th to 37th days, it will be considered as 1.5 months. Similarly, a pawn slip with a 60th to 67th days period is considered as 2.5 months and so on.

R_2.10 : After the first month, if the loan period is after the first seven days of each month, the loan period will be calculated as one full month plus months already passed. For example; on the day of redemption, if a pawn slip has a loan period between 37th to 60th days, it will be considered as 2 months and so on.

R_2.11 : If the calculated loan period is more than 5 months (150 days), it will display an error message saying it is already expired.

R_2.12 : If the redemption date is somehow later than the pawn date, it displays an error message.

Functional Specifications with Pseudo-Code Mapping

Loan Estimation Function

Description

The loan estimation function allows users to estimate the loan amount that can be offered by entering the weight of the jewelry in grams and selecting its purity.

Here is the pseudo-code for the Loan Estimation function.

```
def calculate_loan_estimation(gold_weight, gold_prices, quality):
    initialize price_entry, price, loan_amount;
    purity = f"{quality}K"

    chosen_entry = None
    for price_entry in gold_prices:
        if price_entry['purity'] == purity:
            chosen_entry = price_entry
            break

    price = chosen_entry['price_per_gram_thb']

    if gold_weight <= 0:
        raise ValueError("Gold weight must be greater than zero.")

    loan_amount = gold_weight * price

    return loan_amount
```

Input and Validation

Inputs:

- gold_weight: The weight of the jewelry in grams.
- quality: The purity of gold (e.g., 18, 22, 24).
- gold_prices: A list containing current prices of different gold purities. Will be stored and retrieved from the database in the fully functioning software. So, the user won't actually need to input.

Validation Rules:

The system ensures the weight is greater than zero and the gold price for the selected purity is correctly defined. It also ensures the user can't put in other than numbers into the input field.

The pseudo-code includes input validation to ensure the gold_weight is greater than zero:

```
if gold_weight <= 0:  
    raise ValueError("Gold weight must be greater than zero.")
```

There will be an initial list of gold prices and updated gold prices are validated in a separate gold price update function. Quality will come from a selector with limited options. So, no need to check whether the user will input a valid quality.

These check that the user enters valid inputs for this function, thereby satisfying the requirement for input validation.

Process

1. Retrieve the Current Price

- The function first determines the purity string format (e.g., "18K", "22K", "24K") using the provided `quality` parameter since the purity attribute is stored as a string in the gold prices table.

```
purity = f"{quality}K"
```

- It then searches for the price entry in the gold prices table corresponding to the selected purity:

```
chosen_entry = None  
for price_entry in gold_prices:  
    if price_entry['purity'] == purity:  
        chosen_entry = price_entry  
        break
```

This loop finds the gold price for the specified purity, fulfilling the requirement to retrieve the price per gram.

2. Calculate the Loan Amount

- After validating inputs and retrieving the price, the function calculates the loan amount:

```
loan_amount = gold_weight * price
```

This calculation is based on the current price per gram for the chosen purity and the weight of the jewelry, satisfying the specification to calculate the loan amount based on weight and price.

Output

The function returns the calculated loan amount, which meets the requirement to display the estimated loan amount to the user:

```
return loan_amount
```

Interest Calculation Function

Description

The interest calculation function computes the total interest that a customer must pay to redeem a pawned item, based on the pawn slip ID and predefined rules.

Here is the pseudo-code for the interest calculation function.

The `calculateInterest(pawnID, pawnItems)` function is designed to compute the interest on a pawned item using the pawn slip ID. It calls the function calculateMonth to calculate the period between the pawning date and redeeming date and calls the calculateRate function to determine the interest rate based on the loan amount. The pseudo-codes for both functions are also provided.

```
function calculateInterest(pawnID, pawnItems):
```

```
    pawnItem pawnItem
```

```
    Date today, pawnDate
```

```
    Double diffMonths, rate, interest;
```

```
    pawnItem = pawnItems[pawnID]
```

```
    amount = pawnItem.loanAmount
```

```
    if pawnItem not in pawnItems:
```

```
        print("Pawn ID does not exist")
```

```
        return
```

```
    if pawnItem.loanPaid:
```

```
        print("Loan is already paid")
```

```
        return
```

```
    if amount < 0:
```

```

        print("Loan Amount must be greater than zero")
        return

today = new Date()
pawnDate = new Date(pawnItem.date)
diffMonths = calculateMonth(today, pawnDate)

if diffMonths > 5:
    print("Loan period is longer than 5 months")
    return

rate = calculateRate(amount)
interest = amount * rate * diffMonths
pawnItem.loanPaid = true

return interest

function calculateMonth(today, pawnDate):
    Double diffMonths
    Integer fullMonths, remainingDays
    Integer diffTime, diffDays;

    diffTime = today - pawnDate
    if diffTime < 0:
        print("Today's date must be later than the pawn date")
        return

    diffDays = diffTime / (1000 * 60 * 60 * 24)

    if diffDays <= 30:
        diffMonths = 1
    else:
        fullMonths = floor(diffDays / 30)
        remainingDays = diffDays % 30
        diffMonths = fullMonths
        if remainingDays <= 7:
            diffMonths += 0.5
        else:
            diffMonths += 1

    return diffMonths

```



```

function calculateRate(loanAmount):
    Double rate;

    if loanAmount < 0:
        print("Loan Amount must be greater than zero")
        return 0
    else if loanAmount < 5000:
        rate = 0.04
    else if loanAmount < 10000:
        rate = 0.035
    else:
        rate = 0.03

    return rate

```

Input and Validation

Inputs:

- `pawnID`: The unique ID associated with the pawned item.
- `pawnItems`: In real-case scenarios, this will come from a database table that stores all pawned items' details. So, the user won't be actually providing this input. But in this project for testing purposes, we are using a Json file with a list of pawnItems.

Validation:

- The pseudo-code includes several validation steps to ensure that the input data is correct:

1. Check if Pawn Item Exists in the Json file (Database)

```

if pawnItem not in pawnItems:
    print("Pawn ID does not exist")
    return

```

This ensures that the `pawnID` provided by the user exists in the database, satisfying the requirement to check for a valid pawn slip ID.

2. Check if Loan is Already Paid

```

if pawnItem.loanPaid:
    print("Loan is already paid")
    return

```

This prevents calculation of interest on an already paid pawn slip ensuring a pawn slip won't be calculated twice once marked paid.

3. Check if Loan Amount is Valid

```
if amount < 0:
    print("Loan Amount must be greater than zero")
    return
```

This ensures the loan amount is valid, which is essential for further calculations.

4. Check Loan Period

```
if diffMonths > 5:
    print("Loan period is longer than 5 months")
    return
```

This check confirms that the loan period does not exceed the maximum term of five months, aligning with the requirement.

Process

1. Calculate the Difference in Months

- The function `calculateMonth(today, pawnDate)` is called to determine the number of months between the pawn date and the current date:

```
diffMonths = calculateMonth(today, pawnDate)
```

The detailed logic for calculating the months is handled in the `calculateMonth` function provided above, including handling cases for all possible scenarios.

The logic in the `calculateMonth` function satisfies the specification to compute interest based on specific rules for different redemption periods which are:

- If the pawn date is before the redemption date : invalid
- Within the 30 days : customer has to pay for one full month
- Within the first 7 days for each subsequent month: full months already passed + 0.5months
- Over the period of 7 days from each subsequent month: full months already passed + 1 month

2. Calculate the Interest Rate

- The function `calculateRate(loanAmount)` provided above determines the rate based on the loan amount.

This fulfills the requirement to calculate the interest rate based on predefined loan amount ranges.

3. Calculate the Interest

- The interest is then calculated using the formula:

$\text{interest} = \text{amount} * \text{rate} * \text{diffMonths}$

This calculation satisfies the specification to compute the total interest from the pawn date to the current date using the specified rules.

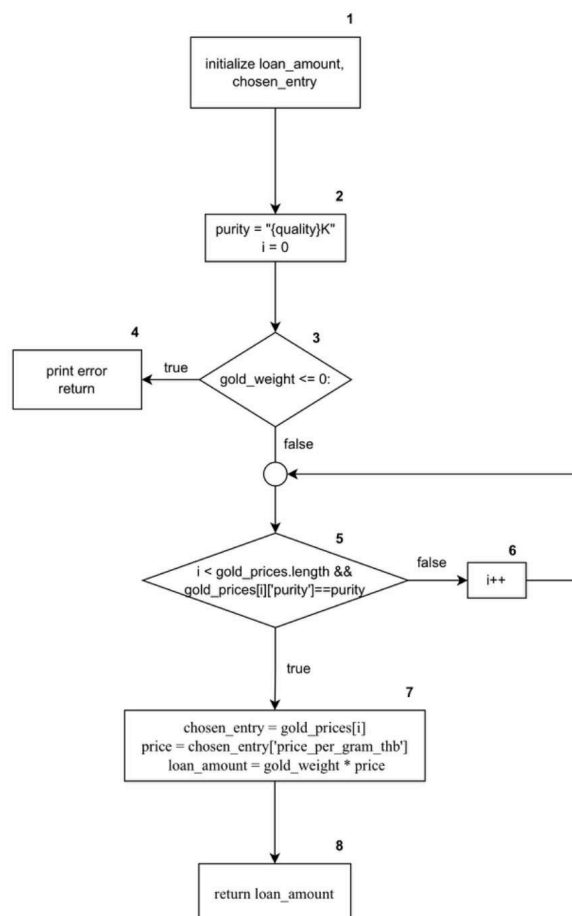
Output

The calculated interest is returned, and considering after the interest is calculated, the customer actually paid the interest, the function marks the loan as paid:

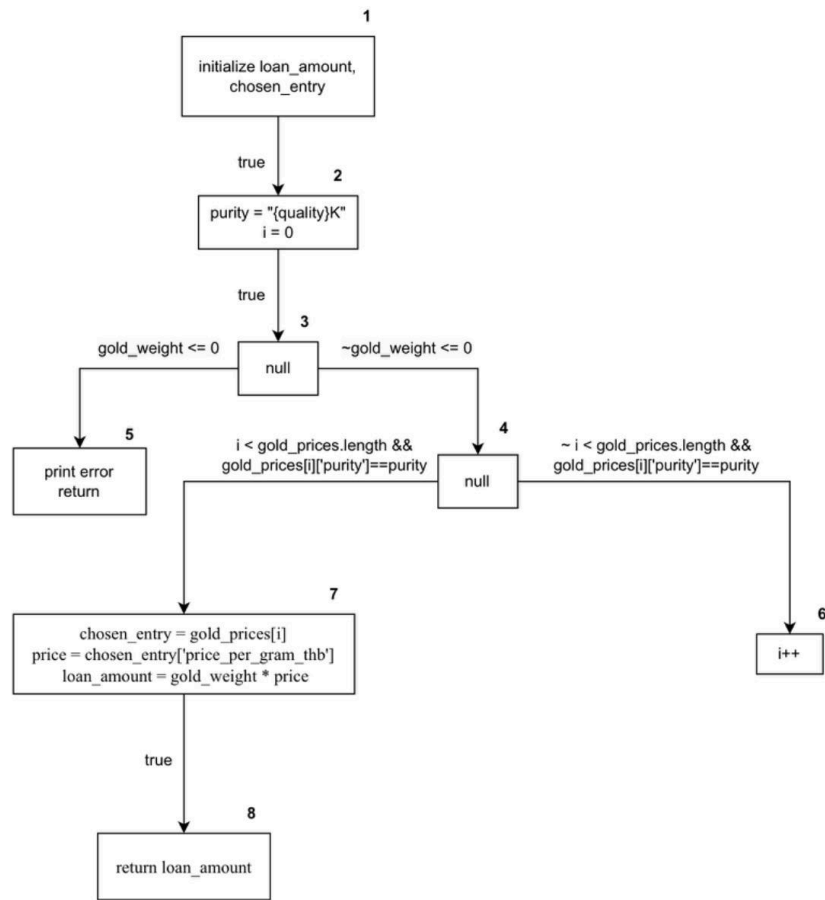
```
pawnItem.loanPaid = true  
return interest
```

This meets the requirement to display the total interest amount to the user and update the status of the loan.

Control Flow Graphs and Data Flow Graphs for Loan Estimation



Control Flow Graph for Loan Estimation



Data Flow Graph for Loan Estimation

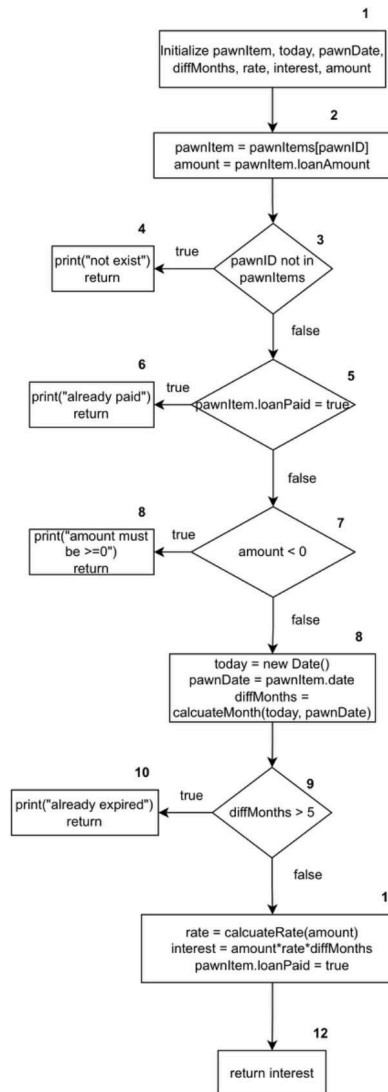
Nodes i	Def (i)	C-use (i)
1	loan_amount, chosen_entry	{ }
2	purity, i	{ }
3	{ }	{ }
4	{ }	{ }
5	{ }	{ }
6	i	{ }
7	chosen_entry, price, loan_amount	gold_prices, i, chosen_entry[], gold_weight, price
8	{ }	loan_amount

Def() and c-use() of Loan Estimation

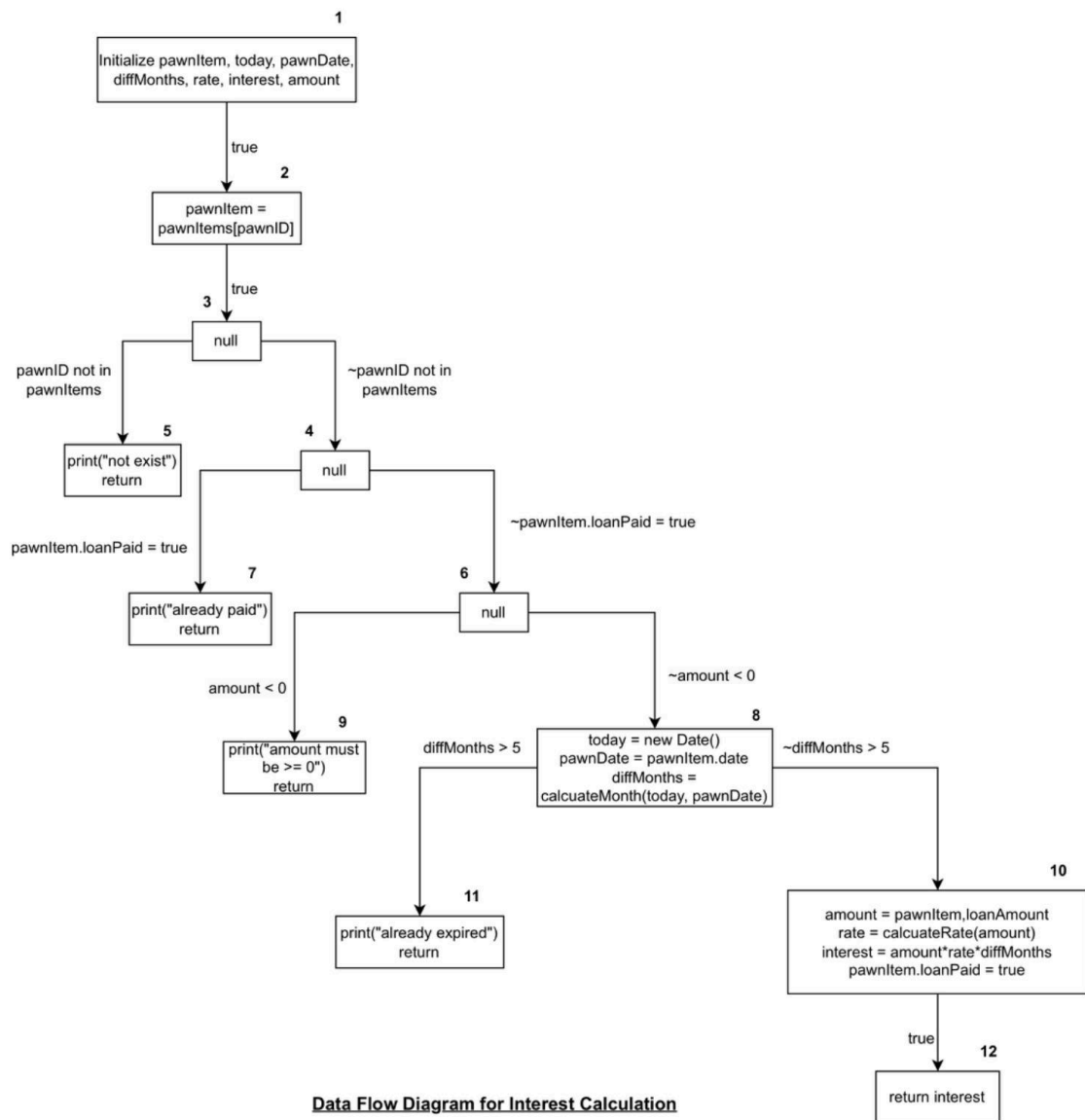
Edges(i, j)	Predicate (i, j)	p-use (i, j)
(1,2)	true	{}
(2,3)	true	{}
(3,4)	$\sim(\text{gold_weight} \leq 0)$	gold_weight
(3, 5)	$\text{gold_weight} \leq 0$	gold_weight
(4,6)	$\sim(i < \text{gold_prices.length} \ \&\& \ \text{gold_prices}[i]['\text{purity}'] == \text{purity})$	gold_price, purity
(4,7)	$i < \text{gold_prices.length} \ \&\& \ \text{gold_prices}[i]['\text{purity}'] == \text{purity}$	gold_price, purity
(7,8)	true	{}

Predicate and p-use() of Loan Estimation

Control Flow Graphs and Data Flow Graphs for Interest Calculation Functions



Control Flow Diagram for Interest Calculation

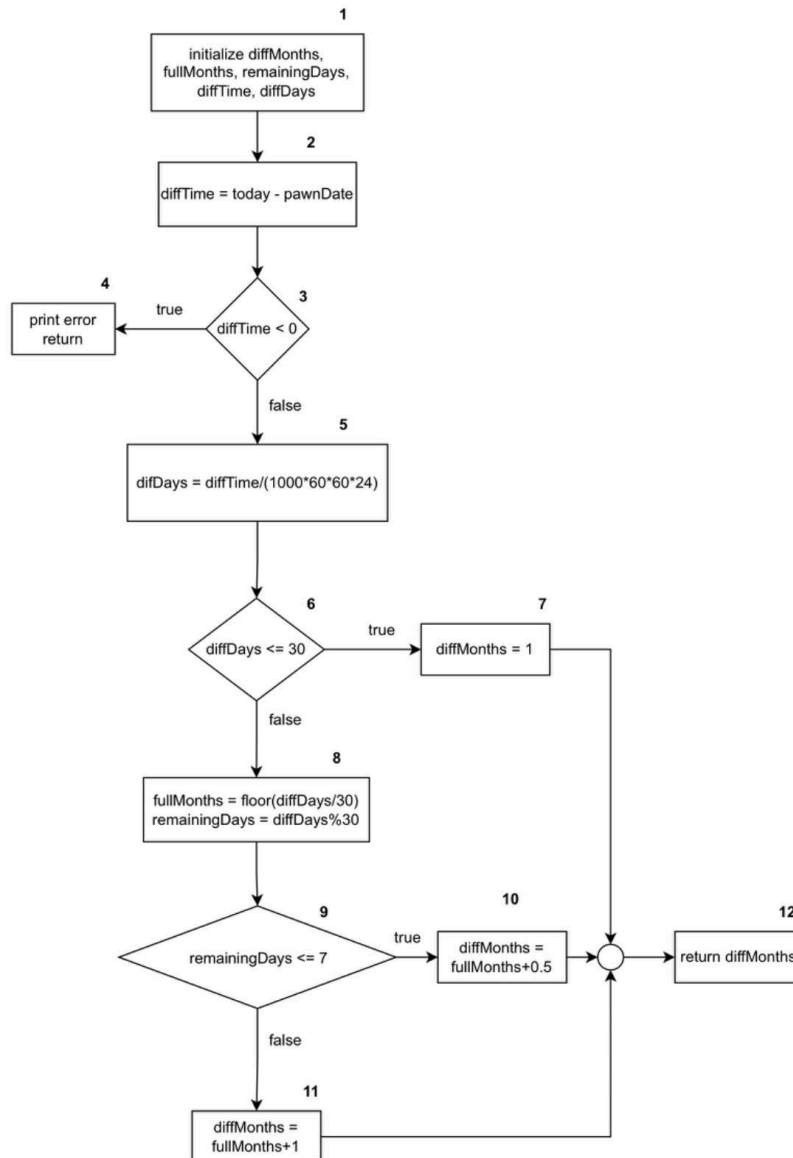


Nodes i	Def (i)	C-use (i)
1	pawnItem, today, pawnDate, diffMonths, rate, interest, amount	{}
2	pawnItem	pawnItems, pawnID
3	{}	{}
4	{}	{}
5	{}	{}
6	{}	{}
7	{}	{}
8	today, pawnDate, diffMonths	Date(), pawnItem.date, calculateMonth(), today, pawnDate
9	{}	{}
10	amount, rate, interest, pawnItem.loanPaid	pawnItem, loanAmount, calculateRate(), amount, rate, diffMonths
11	{}	{}
12	{}	interest

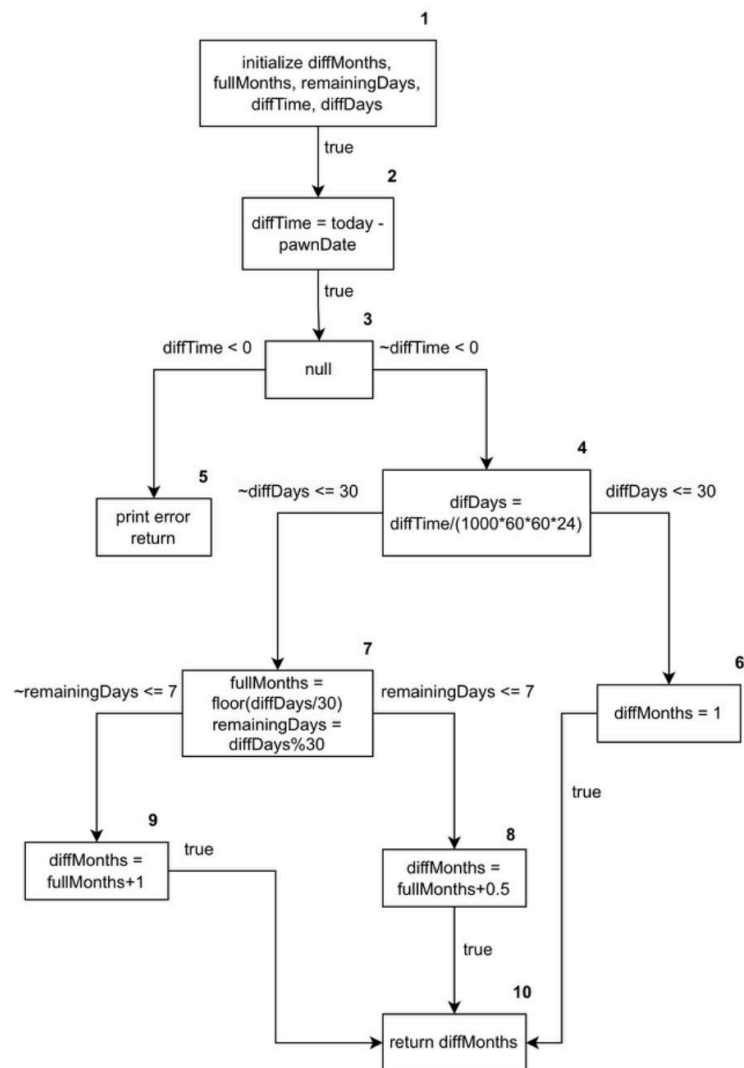
Def() and c-use() of Interest calculation

Edges(i, j)	Predicate (i, j)	p-use (i, j)
(1,2)	true	$\emptyset$
(2,3)	true	$\emptyset$
(3,4)	$\sim(\text{pawnID not in pawnItems})$	<code>pawnID, pawnItems</code>
(3, 5)	<code>pawnID not in pawnItems</code>	<code>pawnID, pawnItems</code>
(4,6)	$\sim(\text{pawnItem.loanPaid} = \text{true})$	<code>pawnItem.loanPaid</code>
(4,7)	<code>pawnItem.loanPaid = true)</code>	<code>pawnItem.loanPaid</code>
(6,8)	$\sim \text{amount} < 0$	<code>amount</code>
(6,9)	<code>amount < 0</code>	<code>amount</code>
(8,10)	$\sim \text{diffMonths} > 5$	<code>diffMonths</code>
(8,11)	<code>diffMonths > 5</code>	<code>diffMonths</code>
(10,12)	true	$\emptyset$

Predicate and p-use() of Interest calculation



Control Flow Diagram for Month Calculation



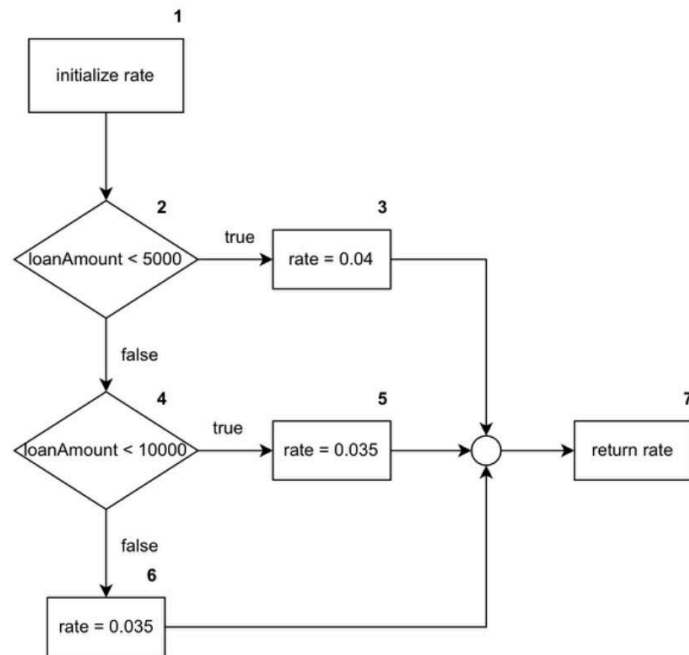
Data Flow Diagram for Month Calculation

Nodes i	Def (i)	C-use (i)
1	diffMonths, fullMonths, remainingDays, diffTime, diffDays	{}
2	diffTime	today, pawnDate
3	{}	{}
4	diffDays	diffTime
5	{}	{}
6	diffMonths	{}
7	fullMonths, remainingDays	diffDays, floor()
8	diffMonths	fullMonths
9	diffMonths	fullMonths
10	{}	diffMonths

Def() and c-use() of Month Calculation

Edges(i, j)	Predicate (i, j)	p-use (i, j)
(1,2)	true	{}
(2,3)	true	{}
(3,4)	~(diffTime < 0)	diffTime
(3,5)	diffTime < 0	diffTime
(4,6)	diffDays < 30	diffDays
(4,7)	~(diffDays < 30)	diffDays
(7,8)	remainingDays <= 7	remainingDays
(7,9)	~(remainingDays <= 7)	remainingDays
(6,10)	true	{}
(8,10)	true	{}
(9,10)	true	{}

Predicate and p-use() of Month Calculation



Control Flow Diagram for Interest Rate Calculation

Nodes i	Def (i)	C-use (i)
1	rate	{}
2	{}	{}
3	rate	{}
4	{}	{}
5	rate	{}
6	rate	{}
7	{}	rate

Def() and c-use() of Interest Rate Calculation

Edges(i, j)	Predicate (i, j)	p-use (i, j)
(1,2)	true	{}
(2,3)	(loanAmount < 5000)	loanAmount
(2,4)	~(loanAmount < 5000)	loanAmount
(4,5)	loanAmount < 10000	loanAmount
(4,6)	~(loanAmount < 10000)	loanAmount
(5,7)	true	{}
(6,7)	true	{}
(3,7)	true	{}

Predicate and p-use() of Interest Rate Calculation

Input domains

Loan Estimation Function

The **Loan Estimation** function allows users to estimate the loan amount based on the weight of the jewelry and by selecting gold purity. The main inputs for this function are gold_weight (weight in grams) and quality (purity of gold).

Inputs and Their Domains:

Gold Weight (gold_weight):

Domain 1: Negative values (e.g., -10, -1.5): Should raise an error (gold_weight <= 0 case).
Domain 2: Zero (e.g., 0): Should raise an error (gold_weight <= 0 case).
Domain 3: Positive values (e.g., 0.1, 1000): Should calculate the correct loan amount.
Domain 4: Non-numeric values (e.g., "abc", "", null): Should raise an error due to type validation.
(In our prototype, we made it to be unable to put in anything other than numbers)

Gold Purity (quality):

Domain 1: Valid purities (e.g., 18, 22, 24): Should correctly fetch the price and calculate the loan amount. These will come from a drop-down selector. So, it is impossible to have other input values.

Gold Prices (gold_prices):

This input comes from a predefined list or database and is not directly provided by the user in the functioning system. Whenever the user tries to change the unit gold price, a separate function for updating gold price which is outside of our scope checks for the validity of this input.

Interest Calculation Function

The **Interest Calculation** function computes the total interest for a pawned item based on the pawn slip ID and predefined rules. The main inputs for this function are pawnID and pawnItems.

Inputs and Their Domains:

Pawn Slip ID (pawnID):

Domain 1: Valid IDs that exist in pawnItems (e.g., ID present in the JSON/database): Should proceed with interest calculation.

Domain 2: Invalid IDs that do not exist in pawnItems (e.g., IDs not present in the JSON/database): Should display "Pawn ID does not exist."

Domain 3: IDs for which the loan is already paid (exists in pawnItems but loanPaid is True): Should display "Loan is already paid."

Domain 4: Non-numeric or malformed IDs (e.g., "", "abc123", null): Should also display "Pawn ID does not exist."

Pawn Items (pawnItems):

This input will also be retrieved from the database in a full-scale system. So, all the input validation before being stored into the database will be handled in a different module (pawn slip registration). So, we'll just include the domains for the attributes that we directly use in the calculation of our scope.

Loan Amount (loanAmount within pawnItems):

Domain 1: Negative values (e.g., -100): Should raise an error "Loan Amount must be greater than zero."

Domain 2: Zero value (e.g., 0): Should not trigger an error (not explicitly stated but is logically valid).

Domain 3: Small positive values (e.g., 1 to 5000): Should use the 0.04 interest rate.

Domain 4: Medium positive values (e.g., 5001 to 9999): Should use the 0.035 interest rate.

Domain 5: Large positive values (e.g., 10000 and above): Should use the 0.03 interest rate.

Date Inputs (today, pawnDate within pawnItems):

Domain 1: Valid dates where today > pawnDate: Should calculate interest based on the rules.

Domain 2: Invalid dates where today <= pawnDate: Should display "Today's date must be later than the pawn date."

Domain 3: Dates resulting in loan periods exceeding 5 months: Should display "Loan period is longer than 5 months."

Software Testing Approach

Testing Strategy: Since this is not a fully functioning system, we are just focusing on the unit testing to validate both functions. Inputs are determined on every possible scenario we can think of for each function.

Tools and Environment: The testing was performed using JavaScript-based simulations in a controlled environment, including input validation and output verification.

Types of Testing Conducted:

Functional Testing: Focused on ensuring each function meets its requirements.

Boundary Testing: Checked edge cases, such as minimum and maximum allowable values.

Validation and Verification Testing: Ensured all inputs, processes, and outputs are correctly handled and displayed.

Test Cases and Scenarios

Loan Estimation Function Test Cases

1. Case 1: Valid Loan Estimation

-TestedID : LE1

- Input: `gold\_weight = 20`, `quality = 18`

-Output: Loan is estimated as per the formula (e.g., loan = `gold\_weight` x `gold\_price`)

- Expected Output : "The shop can offer up to : 32000.00 THB."

2. Case 2: Gold Weight with Floating points

-TestedID : LE2

- Input: `gold\_weight = 13.5`, `quality = 22`

- Output: Loan is estimated as per the formula (e.g., loan = `gold\_weight` x `gold\_price`)

- Expected Output : "The shop can offer up to : 25650.00 THB."

3. Case 3: Gold Weight is Zero

-TestedID : LE3

- Input: `gold\_weight = 0`, `quality = 22`

- Output: Error message indicating that please enter a valid value.

- Expected Output : "Error: Gold weight must be greater than zero."

4. Case 4: Gold Weight is Negative

-**TestedID** : LE4

- **Input**: `gold\_weight = -0.01`, `gold\_quality = 24`

- **Output**: Error message indicating that please enter a valid value.

- **Expected Output** : "Error: Gold weight must be a positive number."

Interest Calculation Function Test Cases

This is an example data (pawnItem object). For interest calculation, only the date and loan amount is used directly and several test cases are listed below.

```
{
  "pawnID": "PAWN023",
  "goldAmount": 7.78,
  "purity": 24,
  "date": "2024-04-12",
  "customerName": "Thet Myat Noe Thwin",
  "address": {
    "city": "Samut Prakarn"
  },
  "note": "The customer paid the interest once. The old slip ID is PAWN007 ",
  "loanPaid": false,
  "loanAmount": 16338
}
```

When testing for pawn slips, there are two possible cases for pawn slip status which are the existence of pawnID and whether the loan is paid or not.

Test cases for status :

Case 1 : Existence of pawn slip

TestID: IC1

Input ⇒ pawn slip ID which doesn't exist in the database

Output ⇒ "Pawn ID does not exist"

TestID: IC2

Input ⇒ pawn slip ID which exists in the database (ID - PAWN025)

Output ⇒ 2268

Case 2 : Status of pawn slips

TestID: IC3

Input ⇒ pawn slip ID which is already paid

Output ⇒ "Loan is already paid"

TestID: IC4

Input ⇒ pawn slip ID which is not paid yet (ID - PAWN025)

Output ⇒ 2268

Case 3 : Invalid loan amounts

TestID: IC5

Input ⇒ pawn slips with impossible loan amounts (ID - PAWN032)

Output ⇒ "How did you take a negative loan"

After these status are checked, the next step is to calculate the interest of the loan. There are several criteria to calculate months based on the pawning date to calculate the interest rate. Pawning date is included in the pawn slip object as "date" attribute. All the possible test cases for pawn date are tested as follows:

Suppose today is **10th September 2024:**

****In all cases, 30 days is calculated as 1 month****

Case 1: Less than 1 month

TestID: IC6

Pawn date ⇒ 10th Sep 2024(today),

Output ⇒ 1 month

TestID: IC7

Pawn date ⇒ 5th Sep 2024 (less than 7 days)

Output ⇒ 1 month

TestID: IC8

Pawn date ⇒ 25th Aug 2024 (more than 7 days less than 30 days)

Output ⇒ 1 month

Number of months for interest calculation : regardless of what date you pawn, the first month will calculate 1 full month

Case 2: Full months

TestID: IC9

Pawn date ⇒ 12th July 2024(60 days)

Output ⇒ 2 months

Number of months for interest calculation : month calculation is based on number of full months

Case 3: Within 5 months with half-month validity

TestID: IC10

Pawn date ⇒ 5th July 2024 (67 days, edge case)
Output ⇒ 2.5 months

TestID: IC11

Pawn date ⇒ 9th May 2024(124 days)
Output ⇒ 4.5 months

Number of months for interest calculation : within 1 week after a full month is valid as half month, therefore, month calculation will be based on the number of full months + half month

Case 4: Within five months with over 7 days range

TestID: IC12

Pawn date ⇒ 4th July 2024(68 days, edge case)
Output ⇒ 3 months

TestID: IC13

Pawn date ⇒ 22nd July 2024(50 days)
Output ⇒ 2 months

Number of months for interest calculation : exceeding 1 week after a full month is not valid as half month anymore. It will be consider as another full month and month calculation will be case on the number of full months

Case 5: Over 5 months (expired)

TestID: IC14

Pawn date ⇒ 12th April 2024(151 days)
Output ⇒ error: "Slip expired"

TestID: IC15

Pawn date ⇒ 12th April 2024(paid)
Output ⇒ error: "Loan already paid"

Number of months for interest calculation : if the pawn date exceeds 150 days, it will raise an "expired" error. However, if this exceeding pawn slip is marked as "paid", our system will notify that this pawn slip is already paid.

Case 6: Pawn date is later than today

TestID: IC16

Pawn date ⇒ 15th Sep 2024
Output ⇒ error: "The pawn date hasn't arrived yet!"

Number of months for interest calculation : raise error (this an impossible case)

Defining Interest Rate

After all the above tests are passed, the system will finally set the interest rate for the slip depending on the loan amount. Loan amount is included in the pawn slip object as “loanAmount” attribute. Test cases for defining interest rate:

Case1: Loan amount less than 5000 THB

TestID: IC17

Loan Amount: 2100 THB

Output ⇒ Interest Rate: 0.04

Case2: Loan amount is more than or equal to 5000 THB but less than 10,000 THB

TestID: IC18

Loan Amount: 7400 THB

Output ⇒ Interest Rate: 0.035

Case3: Loan amount more than or equal to 10,000 THB

TestID: IC19

Loan Amount: 32,200 THB

Output ⇒ Interest Rate: 0.03

Test Results and Analysis

Summary of Test Results: A table of all test cases executed is provided below, indicating whether each test passed or failed, the inputs used, expected outcomes, actual outcomes, and remarks.

Test Case ID	Function	Input Parameters	Expected Output	Actual Output	Status	Requirement
LE1	Loan Estimation	gold_weight = 20, quality = 18	32000	32000	passed	R_1.2
LE2	Loan Estimation	gold_weight = 13.5, quality = 22	25650	25650	passed	R_1.2

LE3	Loan Estimation	gold_weight = 0, quality = 22	Error: "Gold weight must be greater than zero."	Error: "Please enter a valid value."	passed	R_1.1
LE4	Loan Estimation	gold_weight = -0.01, quality = 24	Error: "Gold weight must be a positive number."	Error: "Please enter a valid value."	passed	R_1.1
IC1	Check Slip Status	PAWN999	Error : "Pawn ID does not exist."	Error : "Pawn ID does not exist"	passed	R_2.1
IC2	Check Slip Status	PAWN025	Interest: 2268	Interest: 2268	passed	R_2.2
IC3	Check Slip Status	PAWN024	Error : "Pawn slip already paid."	Error : "Loan is already paid."	passed	R_2.3
IC4	Check Slip Status	PAWN025	Interest: 2268	Interest: 2268	passed	R_2.3
IC5	Month Calculation	PAWN032	Error: "How did you take a negative loan"	Error: "How did you take a negative loan"	passed	R_2.4
IC6	Month Calculation	PAWN030	Month : 1	Month : 1	passed	R_2.8
IC7	Month Calculation	PAWN038	Month : 1	Month : 1	passed	R_2.8
IC8	Month Calculation	PAWN029	Month : 1	Month : 1	passed	R_2.8
IC9	Month Calculation	PAWN037	Months: 2	Months: 2	passed	R_2.10

IC10	Month Calculation	PAWN027	Months: 2.5	Months: 2.5	passed	R_2.9
IC11	Month Calculation	PAWN025	Months: 4.5	Months: 4.5	passed	R_2.9
IC12	Month Calculation	PAWN026	Months :3	Months :3	passed	R_2.10
IC13	Month Calculation	PAWN028	Months: 2	Months: 2	passed	R_2.10
IC14	Month Calculation	PAWN023	Error : "The pawn slip is already expired"	Error : "Loan period is longer than 5 months."	passed	R_2.11
IC15	Month Calculation	PAWN024	Error : "Loan is already paid"	Error : "Loan is already paid"	passed	R_2.3
IC16	Month Calculation	PAWN031	Error: "Pawn date haven't even arrived yet"	Error: "Pawn date haven't even arrived yet"	passed	R_2.12
IC17	Defining Interest Rate	PAWN033	Interest Rate: 0.04	Interest Rate: 0.04	passed	R_2.5
IC18	Defining Interest Rate	PAWN034	Interest Rate: 0.035	Interest Rate: 0.035	passed	R_2.6
IC19	Defining Interest Rate	PAWN035	Interest Rate: 0.03	Interest Rate: 0.03	passed	R_2.7

User Interface Testing

Our pawn shop interface is minimal but functional so that the staff of the shop can use it easily.

Loan Estimation
Pawn Slip Registration
Interest Calculation
Report

Purity	Price per Gram (THB)
18K	1600
22K	1900
24K	2100

Update Gold Price

Gold Quality:

☒ 18 Karat
☐ 22 Karat
☐ 24 Karat

Weight (grams):

Gold Price (THB per gram):

1600

Submit

In our main page you can clearly see the two windows, one for showing gold prices and the other for calculating estimated loan by choosing gold quality and inputting gold weight. Even though, it is outside of our scope, users can update gold price per gram through Update Gold Price button.

Loan Estimation
Pawn Slip Registration
Interest Calculation
Report

Purity	Price per Gram (THB)
18K	1600
22K	1900
24K	2100

Update Gold Price

Gold Quality:

18 Karat

Gold Weight (grams):

10.8

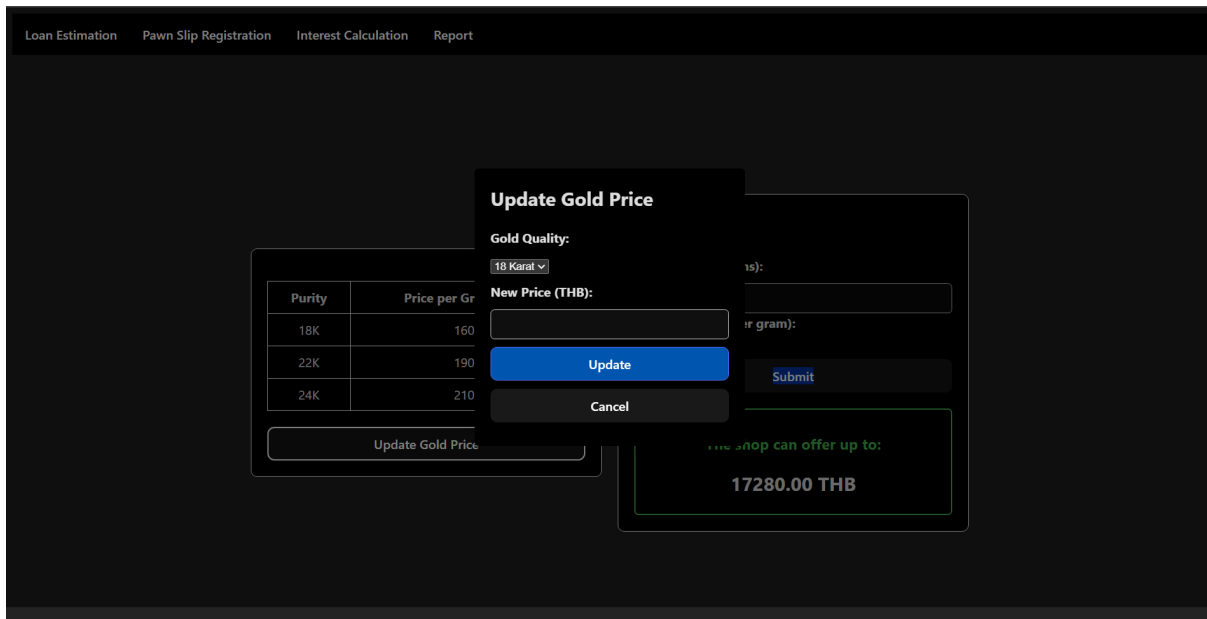
Gold Price (THB per gram):

1600

Submit

The shop can offer up to:
17280.00 THB

Here is what the user will see after choosing gold quality, providing valid gold weight and clicking on submit. The system will calculate estimated loan and displays the result to use.



This is the UI for updating gold price. Again, user can select a quality and put in the new unit price to update it. The update will also immediately reflect to the other part of the system where gold price data is used, like in table display and loan estimation function. The buttons have a hover effect so the users will know that it is clickable and designed with contrasting colors to give out information to our user clearly.

This screenshot shows the 'Update Gold Price' modal with an invalid input. The 'Gold Weight (grams):' field contains '-20', which is highlighted in red. Below the field, a red error message reads 'Please enter a valid value'. The 'Gold Price (THB per gram):' field shows '1600'. The 'Submit' button is highlighted with a blue border.

This screenshot shows the 'Update Gold Price' modal with an invalid input. The 'New Price (THB):' field contains '-9.3', which is highlighted in red. Below the field, a red error message reads 'Please enter a valid value'. The 'Update' and 'Cancel' buttons are visible at the bottom.

These are what the user will see when they try to put in invalid gold prices or gold weights.

The screenshot shows a web application with a dark theme. At the top, there is a navigation bar with four links: "Loan Estimation", "Pawn Slip Registration", "Interest Calculation", and "Report". The "Interest Calculation" link is highlighted. Below the navigation bar, there is a central form titled "Interest Calculation". The form contains a label "Pawn ID:" followed by a text input field. Below the input field is a button labeled "Calculate".

Our interest calculation page has consistent design with our main page with it being minimal and displaying only the function our user requires.

The screenshot shows the same web application as the previous one, but now the "Interest Calculation" form is populated with data. The "Pawn ID:" field contains the text "PAWN034". Below this is a "Calculate" button. Below the button, the form displays the following information: "Loan Amount: 21600 THB", "Interest Rate:" followed by a text input field containing "0.03", "Months: 2", "Total Interest: 1296.00 THB", and "Total Payment: 22896 THB". At the bottom of the form is a blue button labeled "Loan Paid".

The button is consistent with the main page. When the user provides a valid pawn slip ID, it calculates the interest according to the rules and displays the interest rate used in the calculation, the calculated loan period, the interest and total payment.

Interest Calculation

Pawn ID:

PAWN000

Calculate

Pawn ID does not exist

Interest Calculation

Pawn ID:

PAWN024

Calculate

Loan is already paid

Interest Calculation

Pawn ID:

PAWN023

Calculate

The pawn slip is already expired

Interest Calculation

Pawn ID:

PAWN031

Calculate

Pawn date haven't even arrived yet!!

Interest Calculation

Pawn ID:

PAWN032

Calculate

How did you take a negative loan?!!

There are the error messages the user would see when the provided pawn slip ID does not exist in the database or when the pawn slip is already marked as paid or when the pawn slip has expired or when the pawn date is later than the redemption date or when the loan amount is invalid.

The following are the other modules that we can integrate into the pawn shop system but not in our scope.

Loan Estimation

Pawn Slip Registration

Interest Calculation

Report

Pawn Slip Registration

Pawn ID:

PAWN004

Gold Amount:

Purity:

Date:

mm/dd/yyyy

Customer Name:

City:

Loan Estimation

Pawn Slip Registration

Interest Calculation

Report

Daily Report

Total Loans Amount (THB)	Total Interest (THB)	Loans Left	Total Number of Clients	Total Profit (THB)	Bad Debt (THB)
150,000.00	5,000.00	3	7	7500.00	2,500.00

Weekly Report

Total Loans Amount (THB)	Total Interest (THB)	Loans Left	Total Number of Clients	Total Profit (THB)	Bad Debt (THB)
1,000,000.00	35,000.00	15	45	50,000.00	15,000.00

Monthly Report

Total Loans Amount (THB)	Total Interest (THB)	Loans Left	Total Number of Clients	Total Profit (THB)	Bad Debt (THB)
4,200,000.00	150,000.00	60	180	200,000.00	50,000.00

The user interface for our pawn shop system is very simple and functional so that the users can quickly input and obtain information, accelerating workflow which increases productivity.

Conclusion

The software testing for the **Loan Estimation** and **Interest Calculation** functions of the pawn shop system successfully validated the correctness, reliability, and performance of these functionalities. All test cases passed, demonstrating that the functions meet the specified requirements. Future improvements may include

optimizing the algorithms for better performance and adding additional features such as more complex interest calculation rules.